LAPORAN TUGAS 2

Sistem Informasi Manajemen Data Mahasiswa (SIMADA)

Penerapan Teknologi Internet · 2026/2027 Gasal



Nama	: Dziban Arkan
NIM	: 77961325
Kelas	: IF-3
Angkatan	: 2024
Program Studi	: Teknik Informatika
Dosen	: Donny Reza, M.Kom

PROGRAM STUDI TEKNIK INFORMATIKA
FAKULTAS TEKNIK DAN ILMU KOMPUTER
UNIVERSITAS KOMPUTER INDONESIA
2026

1. Deskripsi Aplikasi

SIMADA (Sistem Informasi Manajemen Data Mahasiswa) adalah aplikasi web sederhana untuk mengelola data mahasiswa: menambah, menampilkan, mengubah, dan menghapus data (CRUD). Aplikasi dibangun menggunakan **PHP murni** tanpa framework dan basis data **MySQL/MariaDB** yang diakses melalui **PDO** (PHP Data Objects). Seluruh halaman pengelolaan data dilindungi oleh sistem login berbasis `$_SESSION`, dan antarmuka dapat ditampilkan dalam **Bahasa Indonesia** maupun **Bahasa Inggris**.

1.1 Teknologi yang Digunakan

Komponen	Keterangan
Bahasa pemrograman	PHP 8 (murni, tanpa framework)
Basis data	MySQL / MariaDB — database `db_kampus`
Akses database	PDO dengan *prepared statement* dan mode error `ERRMODE_EXCEPTION`
Antarmuka	HTML5 + CSS murni (`assets/css/style.css`)
Server lokal	XAMPP (Apache + MySQL) atau `php -S localhost:8000`

1.2 Pemenuhan Spesifikasi Tugas

Kebutuhan	Implementasi	File
Form input metode POST	Form tambah/ubah data dan form login memakai `method="POST"`	includes/form_mahasiswa.php, login.php
Validasi dengan if-else / switch	`switch ($field)` untuk tiap kolom, di dalamnya `if / elseif` untuk cek kosong, format, rentang, dan duplikat NIM	includes/validasi.php
Sanitasi input	`htmlspecialchars()` saat menampilkan data, `strip_tags()`, `filter_var()` dengan `FILTER_SANITIZE_EMAIL`, `FILTER_SANITIZE_NUMBER_INT`, `FILTER_VALIDATE_EMAIL`, `FILTER_VALIDATE_INT`	includes/validasi.php, includes/init.php
Koneksi database	`new PDO($dsn, $username, $password)` di dalam `try-catch`, file terpisah dan dipanggil dengan `require`	config/koneksi.php
Create	`INSERT INTO mahasiswa ...` dengan named parameter	tambah.php
Read	`SELECT` + `fetchAll()` ditampilkan dalam tabel HTML, plus statistik dan pencarian (GET)	index.php
Update	Form terisi data lama berdasarkan identitas unik `id`; `UPDATE ... WHERE id = :id`	edit.php
Delete	`DELETE FROM mahasiswa WHERE id = :id`, dengan konfirmasi dan `rowCount()`	hapus.php
Login dengan $_SESSION	Login menyimpan `$_SESSION["user_id"]`; halaman CRUD memanggil `auth.php` yang mengarahkan ke login bila session tidak ada	login.php, includes/auth.php, logout.php
Dual bahasa di $_SESSION	Pilihan ID/EN disimpan di `$_SESSION["lang"]`; kamus berupa array asosiatif	set_lang.php, includes/init.php, lang/id.php, lang/en.php

2. Struktur Proyek dan Rancangan Database

2.1 Struktur Folder

```
sim-mahasiswa/
├── index.php              READ   - daftar data mahasiswa (dilindungi login)
├── tambah.php             CREATE - form & proses tambah data
├── edit.php               UPDATE - form & proses ubah data berdasarkan id
├── hapus.php              DELETE - proses hapus data (POST)
├── login.php              Halaman login
├── logout.php             Keluar (session_unset + session_destroy)
├── set_lang.php           Ganti bahasa antarmuka (ID / EN)
├── config/koneksi.php     Koneksi database PDO
├── includes/
│   ├── init.php           session_start(), pemilihan bahasa, fungsi bantu, token CSRF
│   ├── auth.php           Proteksi halaman CRUD
│   ├── validasi.php       Sanitasi & validasi data form
│   ├── form_mahasiswa.php Form yang dipakai bersama tambah & edit
│   ├── header.php / footer.php
├── lang/id.php, lang/en.php   Kamus bahasa (array asosiatif)
├── assets/css/style.css
└── database/db_kampus.sql     Skrip ekspor database
```

2.2 Rancangan Database `db_kampus`

Tabel **mahasiswa** mengikuti contoh pada materi PDO (kolom `id, nim, nama, email, jurusan, angkatan`) dan ditambah kolom `jenis_kelamin` serta `alamat` agar form dapat memakai elemen radio dan textarea.

Tabel mahasiswa	
id	INT, PK, AUTO_INCREMENT
nim	VARCHAR(10), NOT NULL, UNIQUE
nama	VARCHAR(100), NOT NULL
email	VARCHAR(100), NOT NULL
jenis_kelamin	ENUM('L','P'), NOT NULL
jurusan	VARCHAR(100), NOT NULL
angkatan	YEAR, NOT NULL
alamat	TEXT, NULL
created_at / updated_at	TIMESTAMP

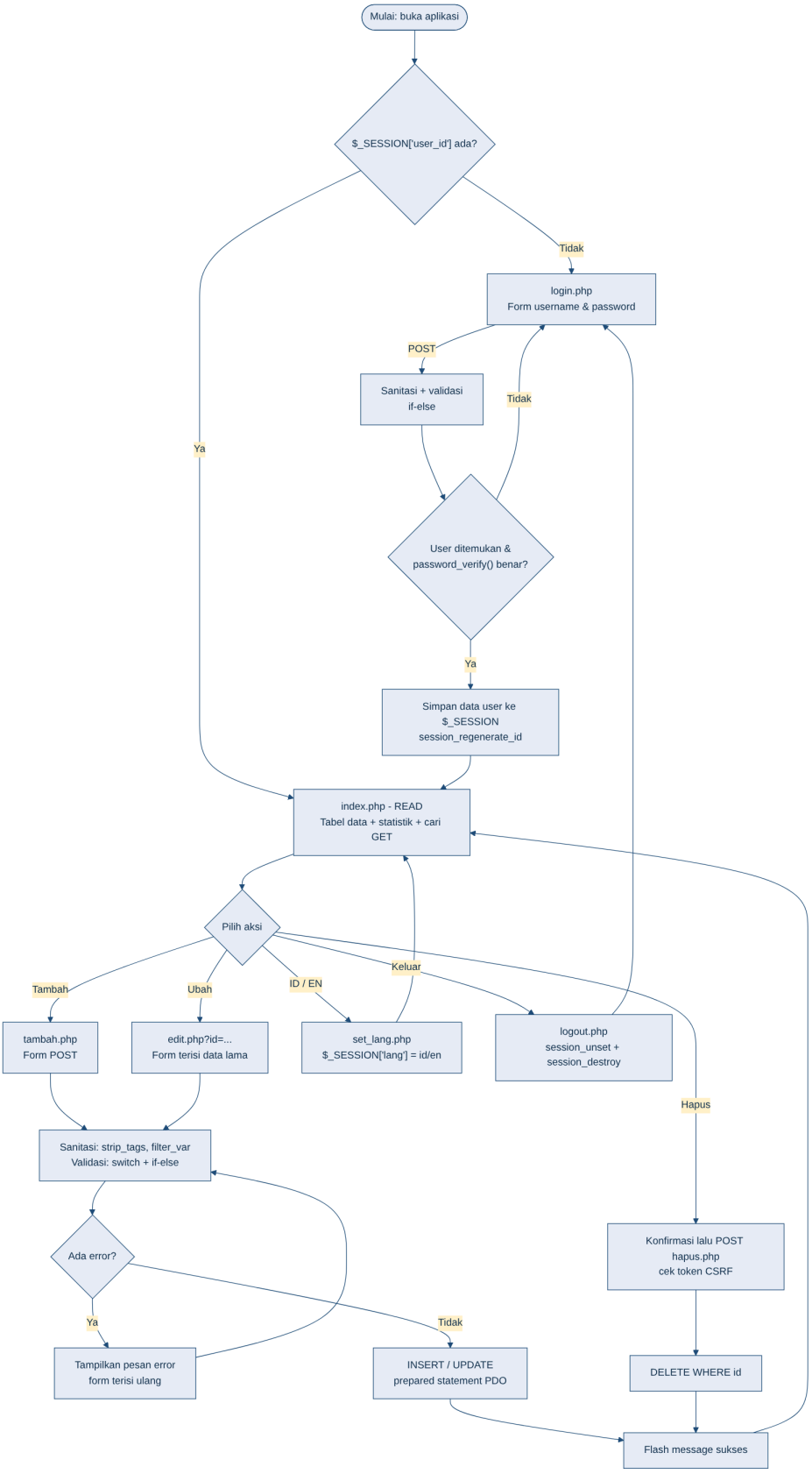
Tabel users	
id	INT, PK, AUTO_INCREMENT
username	VARCHAR(50), UNIQUE
password	VARCHAR(255) — hash dari `password_hash()`
nama_lengkap	VARCHAR(100)

Akun login default: username `admin`, password `admin123`. Password tidak disimpan sebagai teks biasa, melainkan sebagai hash bcrypt.

2.3 Cara Menjalankan

1. Salin folder `sim-mahasiswa` ke `C:\xampp\htdocs\`, lalu jalankan Apache dan MySQL di XAMPP.
2. Buka `http://localhost/phpmyadmin` → **Import** → pilih `database/db_kampus.sql` → **Go**.
3. Jika MySQL memakai password, sesuaikan `$password` di `config/koneksi.php`.
4. Buka `http://localhost/sim-mahasiswa/` dan login dengan akun di atas.

3. Alur Program



Gambar 3.1 Diagram alur program SIMADA

Penjelasan Alur Program

3.1 Inisialisasi setiap halaman

Setiap halaman diawali dengan `require "includes/init.php"`. File ini memanggil `session_start()` sebelum ada output, memuat koneksi PDO dari `config/koneksi.php`, lalu memilih kamus bahasa sesuai `$_SESSION["lang"]` menggunakan `switch`. Halaman yang hanya boleh diakses setelah login juga memanggil `includes/auth.php`:

```
if (!isset($_SESSION["user_id"])) {
    set_flash("error", t("flash_login_dulu"));
    redirect("login.php");
}
```

3.2 Login dan Logout

Form login dikirim dengan metode POST ke file yang sama. Input dibersihkan dengan `htmlspecialchars()` dan `trim()`, lalu dicek kelengkapannya dengan `if-else`. Data user dicari dengan *prepared statement*, dan password dicocokkan memakai `password_verify()`. Jika benar, `session_regenerate_id(true)` dipanggil lalu `user_id`, `username`, dan `nama_lengkap` disimpan ke `$_SESSION`. Logout memanggil `session_unset()` dan `session_destroy()`, kemudian session baru dibuat hanya untuk mempertahankan pilihan bahasa.

3.3 Create dan Update

Form tambah (`tambah.php`) dan ubah (`edit.php?id=…`) memakai komponen form yang sama. Saat form dikirim, data melewati tiga tahap:

1. **Sanitasi** — `sanitasi_mahasiswa()` membuang spasi berlebih, tag HTML (`strip_tags`), karakter tidak sah pada email (`FILTER_SANITIZE_EMAIL`) dan angka (`FILTER_SANITIZE_NUMBER_INT`).
2. **Validasi** — `validasi_mahasiswa()` memeriksa setiap kolom dengan `switch` dan `if-elseif`. Contoh potongan kodenya:

```
switch ($field) {
    case "nim":
        if ($nilai === "") {
            $error[$field] = t("err_nim_kosong");
        } elseif (!preg_match('/^[0-9]{8,10}$/', $nilai)) {
            $error[$field] = t("err_nim_format");
        } else { /* cek NIM duplikat dengan prepared statement */ }
        break;
    case "email":
        if ($nilai === "") { $error[$field] = t("err_email_kosong"); }
        elseif (filter_var($nilai, FILTER_VALIDATE_EMAIL) === false) {
            $error[$field] = t("err_email_format");
        }
        break;
    ...
}
```

3. **Simpan** — jika array `$error` kosong, data disimpan dengan `INSERT` atau `UPDATE ... WHERE id = :id` memakai *named parameters*. Setelah berhasil, pesan sukses disimpan di session (*flash*

message) dan pengguna diarahkan ke halaman data. Jika ada error, form ditampilkan lagi lengkap dengan pesan error di setiap kolom dan isian sebelumnya.

3.4 Read dan Pencarian

`index.php` mengambil seluruh data dengan `SELECT * FROM mahasiswa ORDER BY nama ASC` dan `fetchAll(PDO::FETCH_ASSOC)`, lalu menampilkannya dalam tabel HTML dengan `foreach`. Setiap nilai ditampilkan melalui `htmlspecialchars()` untuk mencegah XSS. Kotak pencarian memakai metode **GET** (sesuai materi: GET cocok untuk pencarian/filter) dan query `LIKE` dengan prepared statement.

3.5 Delete

Tombol **Hapus** berada di dalam form POST kecil berisi `id` dan token CSRF. Browser menampilkan konfirmasi terlebih dahulu. `hapus.php` hanya menerima POST dengan token yang valid, kemudian menjalankan `DELETE FROM mahasiswa WHERE id = :id` dan memeriksa `rowCount()` untuk memastikan data benar-benar terhapus.

```
$stmt = $pdo->prepare("DELETE FROM mahasiswa WHERE id = :id");
$stmt->execute([":id" => $id]);
if ($stmt->rowCount() > 0) {
    set_flash("success", sprintf(t("flash_hapus_ok"), $mhs["nama"]));
}
```

3.6 Fitur Dual Bahasa

Tombol **ID / EN** pada bagian atas mengarah ke `set_lang.php?lang=…`. File tersebut menyimpan pilihan ke `$_SESSION["lang"]` lalu kembali ke halaman sebelumnya. Karena nilai disimpan di session, bahasa tetap sama ketika pengguna berpindah halaman, bahkan setelah logout. Teks antarmuka diambil dari array asosiatif di `lang/id.php` dan `lang/en.php` melalui fungsi `t()`:

```
// includes/init.php
switch ($_SESSION["lang"]) {
    case "en": $lang = require "lang/en.php"; break;
    default:   $lang = require "lang/id.php"; break;
}
function t($key) { global $lang; return $lang[$key] ?? $key; }

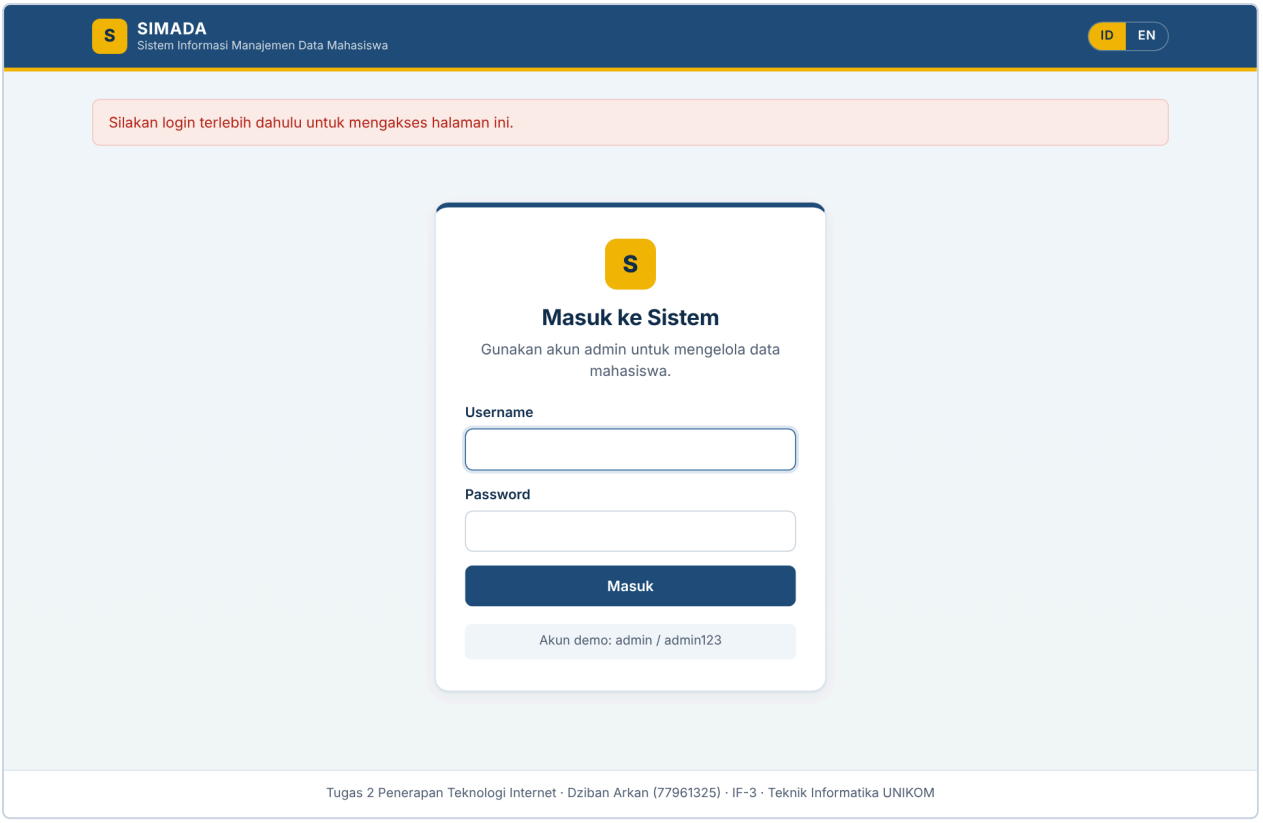
// lang/en.php                          // lang/id.php
"btn_simpan" => "Save",                 "btn_simpan" => "Simpan",
```

3.7 Ringkasan Keamanan

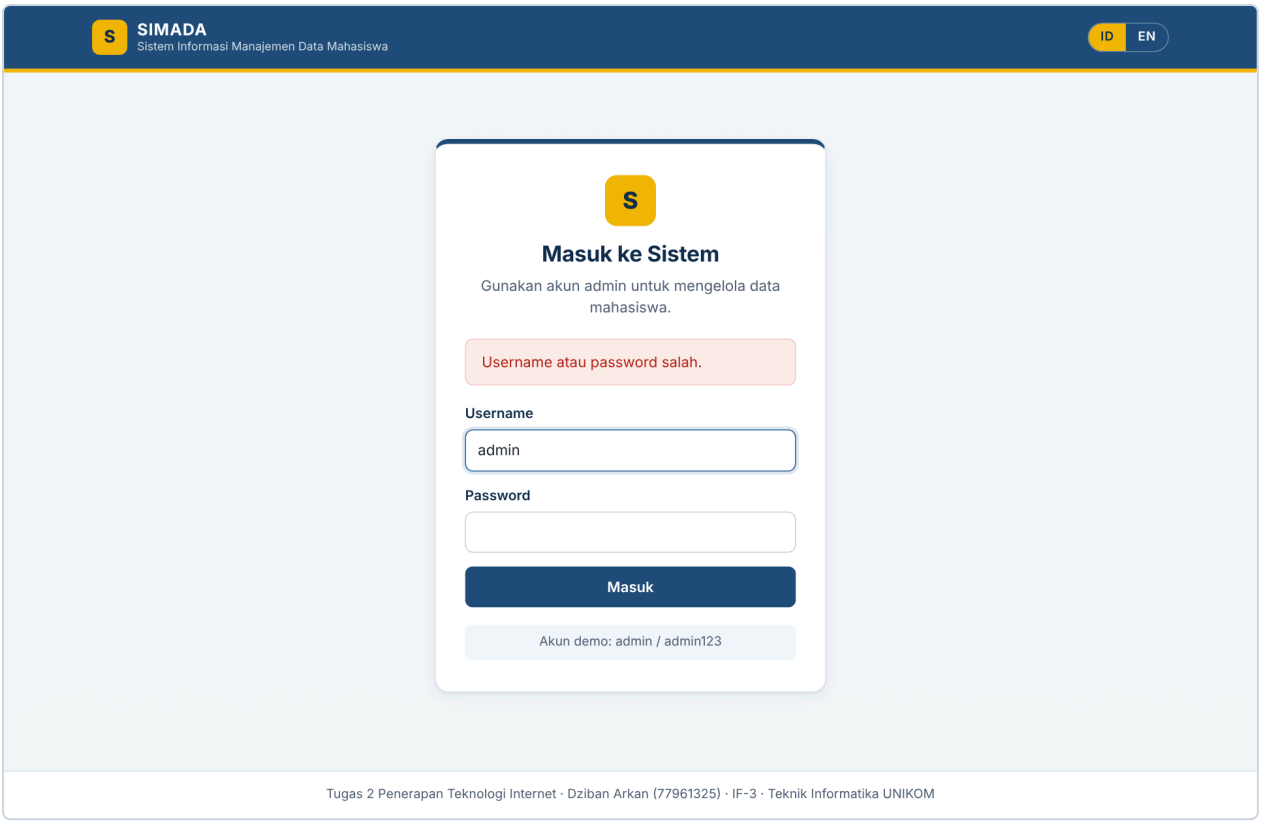
Ancaman	Penanganan
SQL Injection	Semua query dengan input pengguna memakai *prepared statement* PDO
Cross-Site Scripting	`htmlspecialchars()` saat output; `strip_tags()` dan `filter_var()` saat input
Akses tanpa login	`auth.php` memeriksa `$_SESSION["user_id"]` di setiap halaman CRUD
Pencurian password	Password disimpan sebagai hash (`password_hash` / `password_verify`)
Session fixation	`session_regenerate_id(true)` setelah login berhasil
CSRF / hapus lewat link	Token acak di setiap form POST; hapus data hanya lewat POST

4. Tangkapan Layar Antarmuka

4.1 Sistem Otentikasi Login

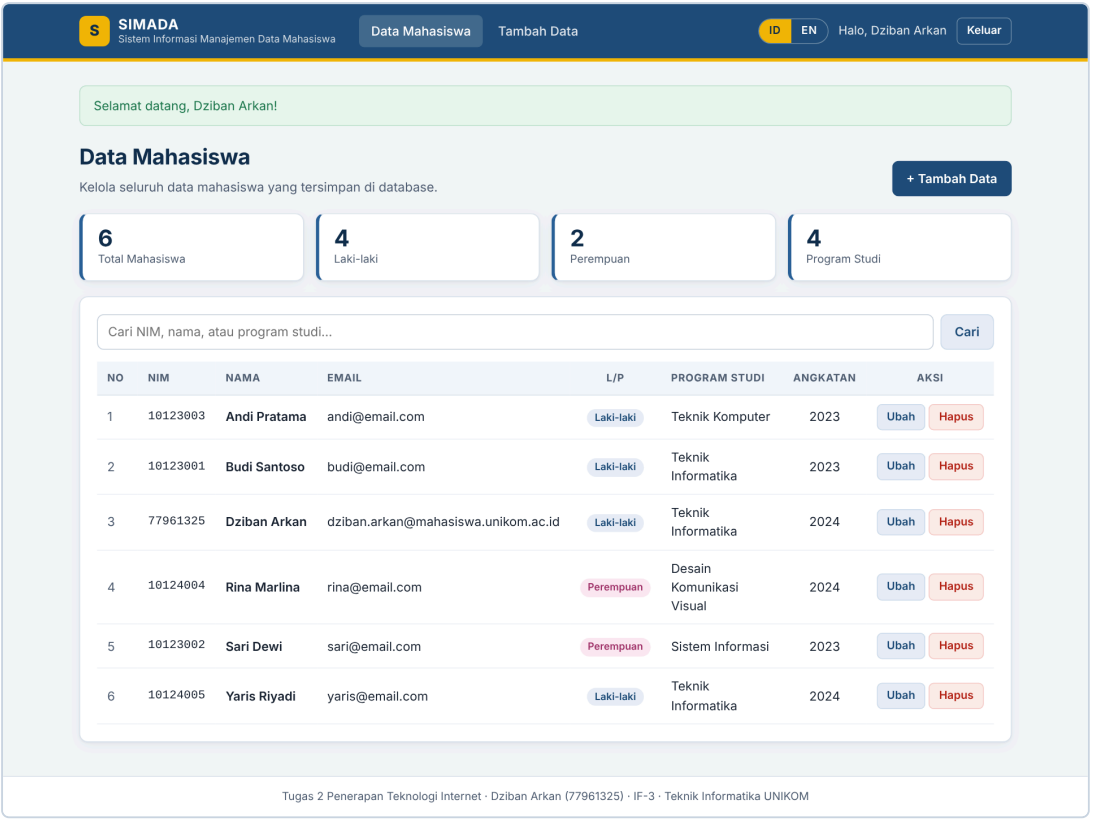


Gambar 4.1 Membuka `index.php` tanpa login langsung diarahkan ke halaman login dengan pesan peringatan (proteksi `$_SESSION`).

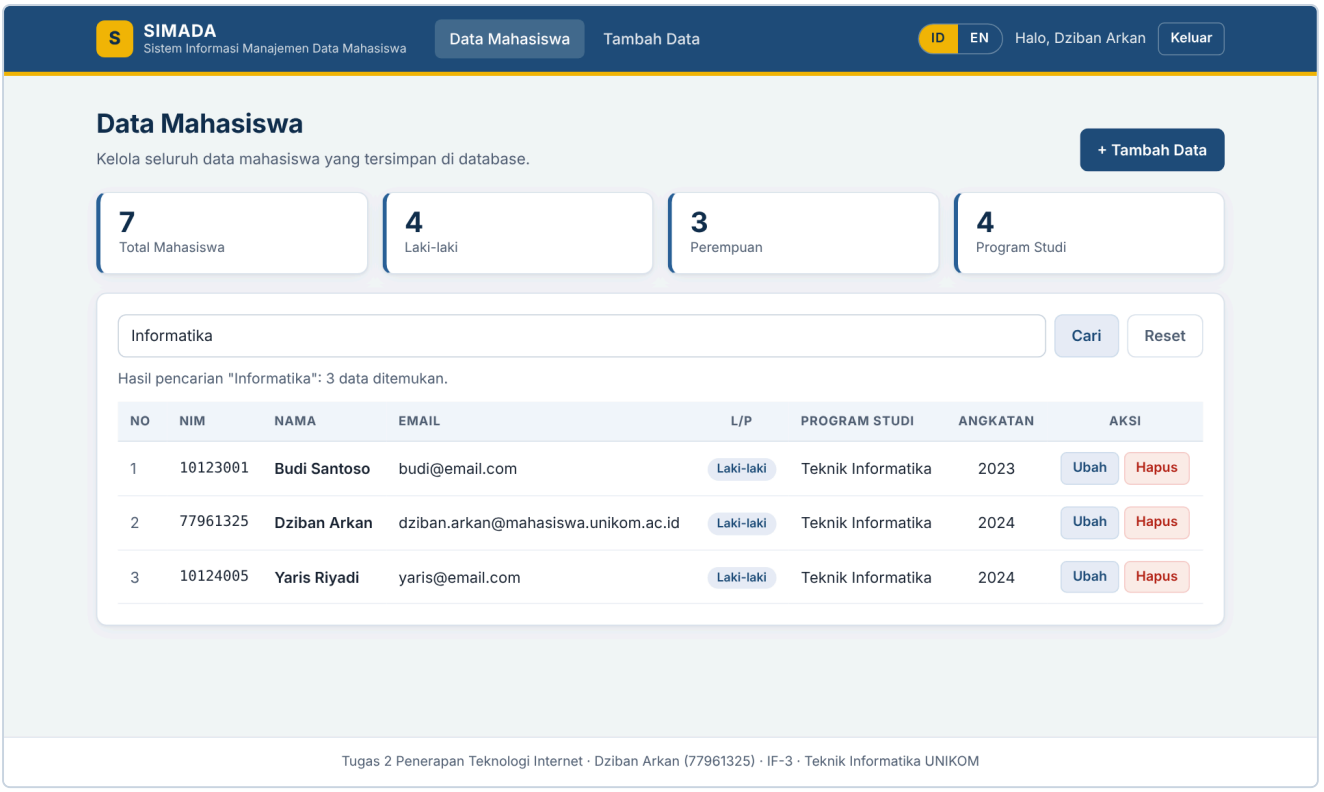


Gambar 4.2 Login gagal karena password salah.

4.2 Read — Daftar Data Mahasiswa

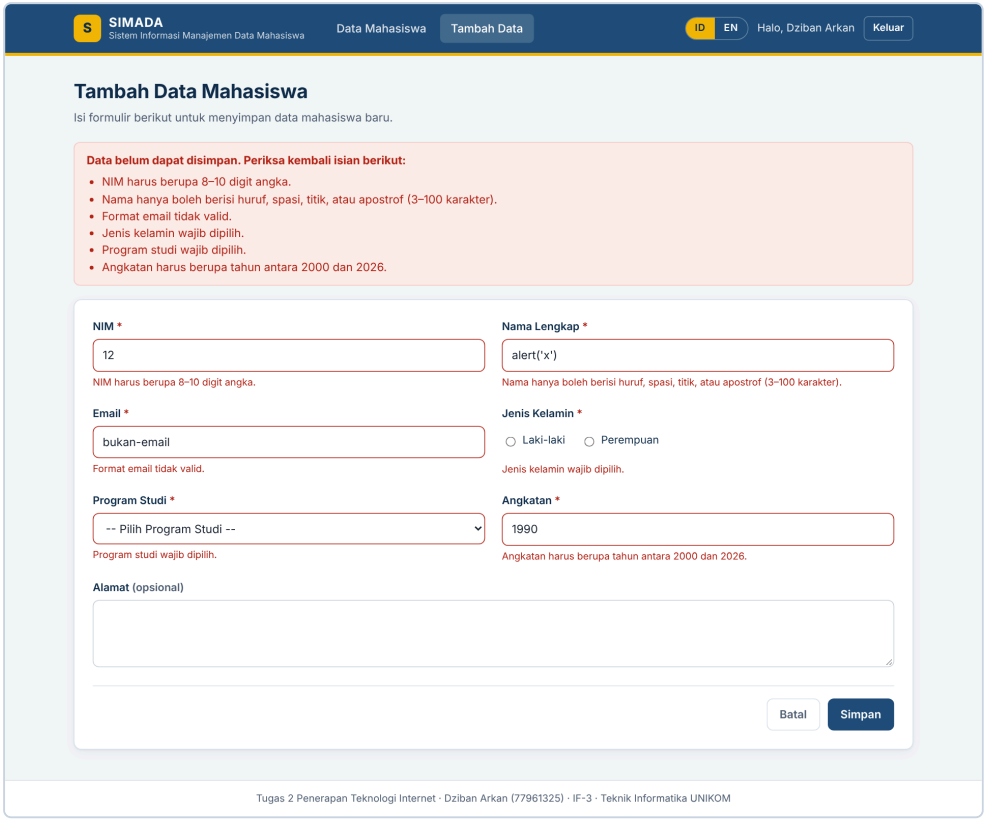


Gambar 4.3 Setelah login berhasil: pesan selamat datang, ringkasan statistik, dan seluruh data mahasiswa dari database dalam tabel HTML.

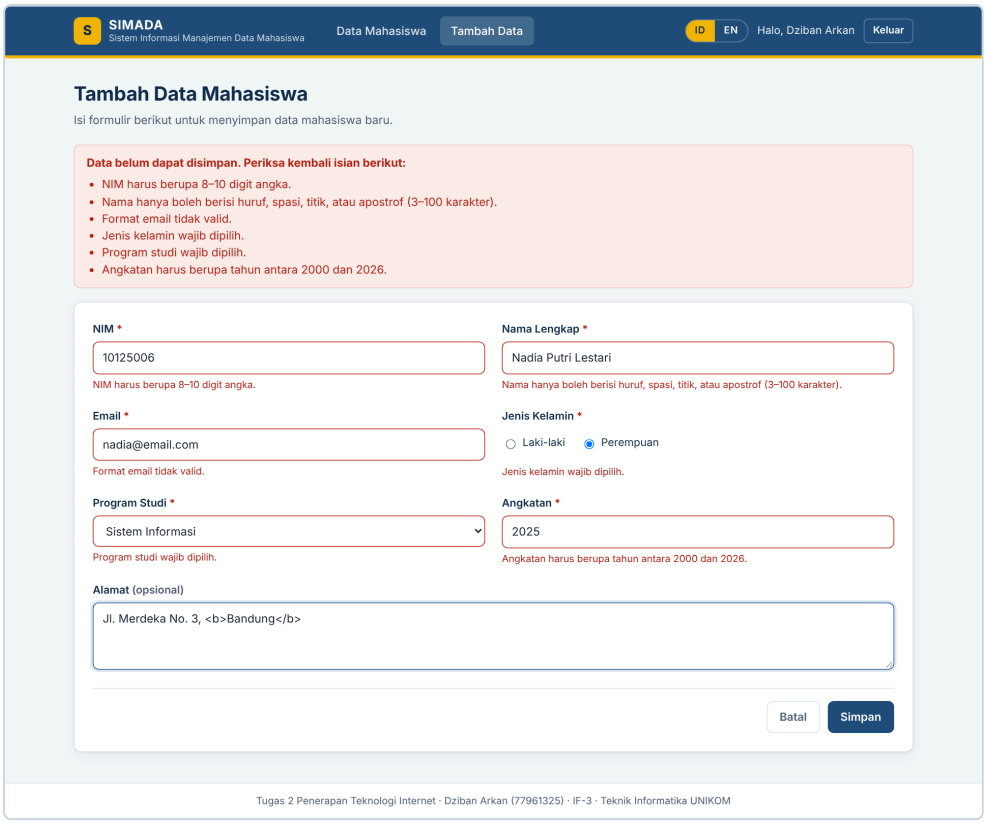


Gambar 4.4 Pencarian dengan metode GET (`index.php?cari=Informatika`).

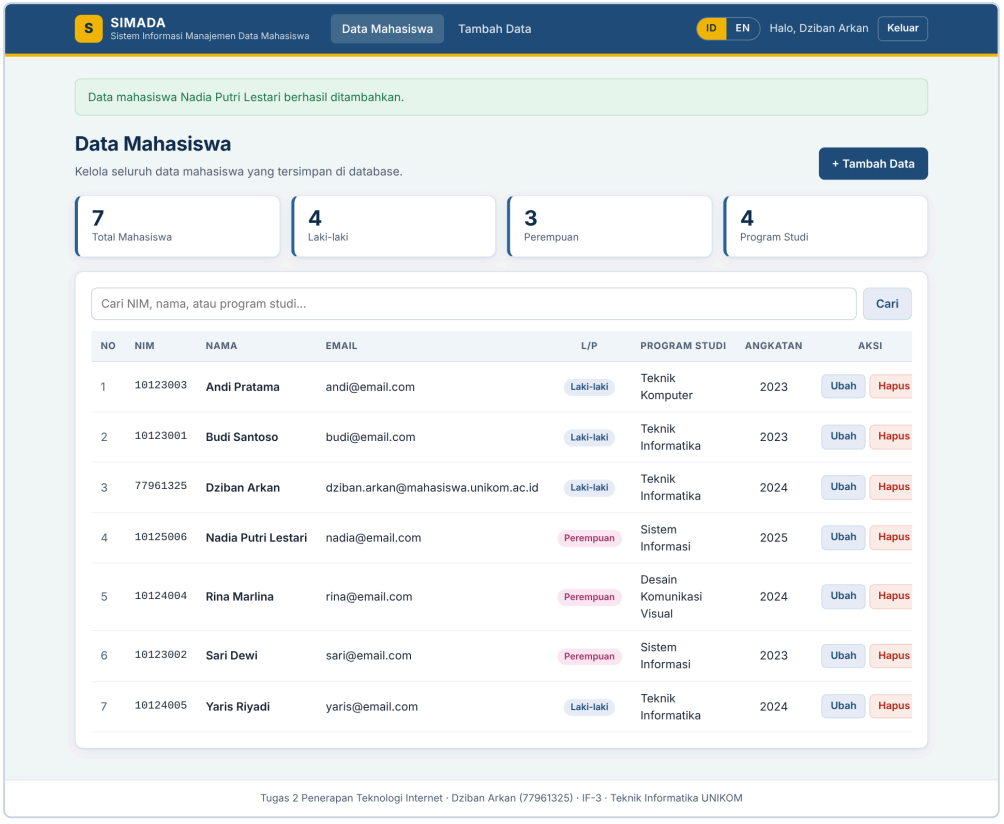
4.3 Create — Tambah Data dan Validasi



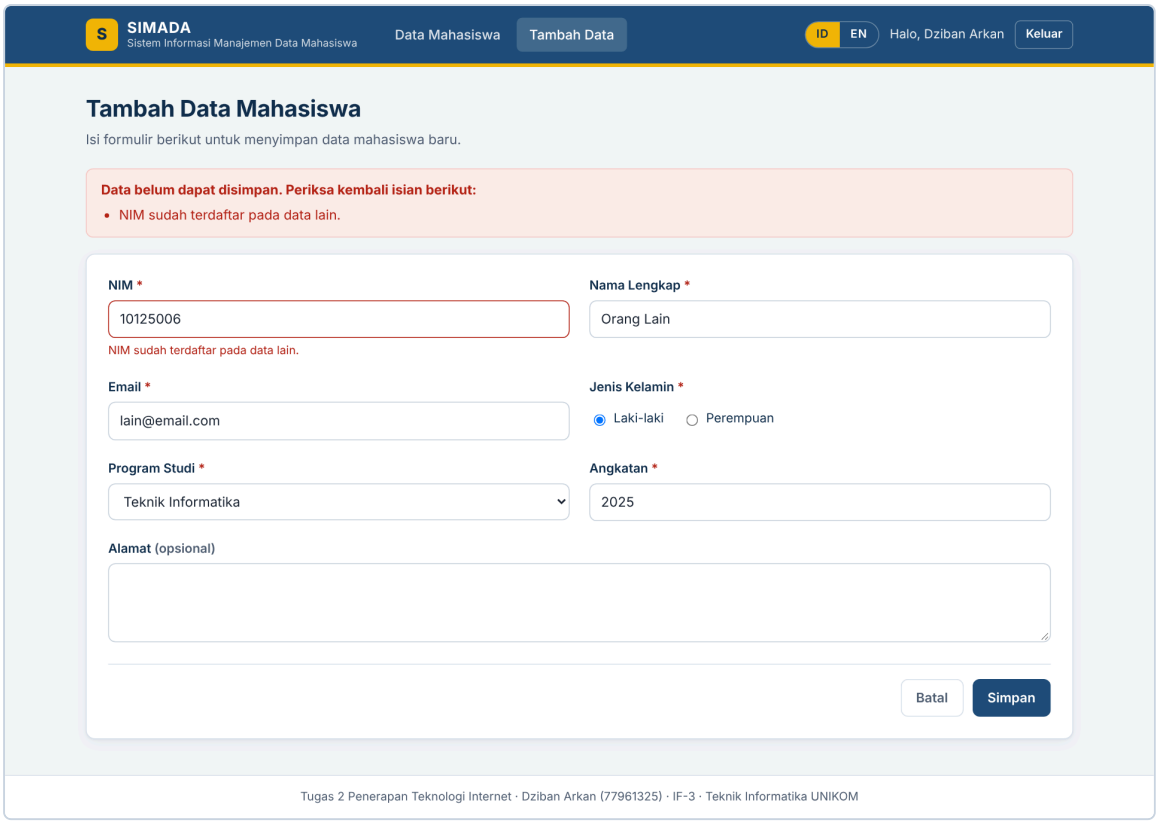
Gambar 4.5 Validasi gagal: NIM bukan angka, nama berisi `<script>` (tag dibuang oleh `strip_tags` lalu ditolak validasi), email salah format, jenis kelamin dan program studi kosong, angkatan di luar rentang.



Gambar 4.6 Form tambah data diisi lengkap (input text, email, radio, select, number, textarea).

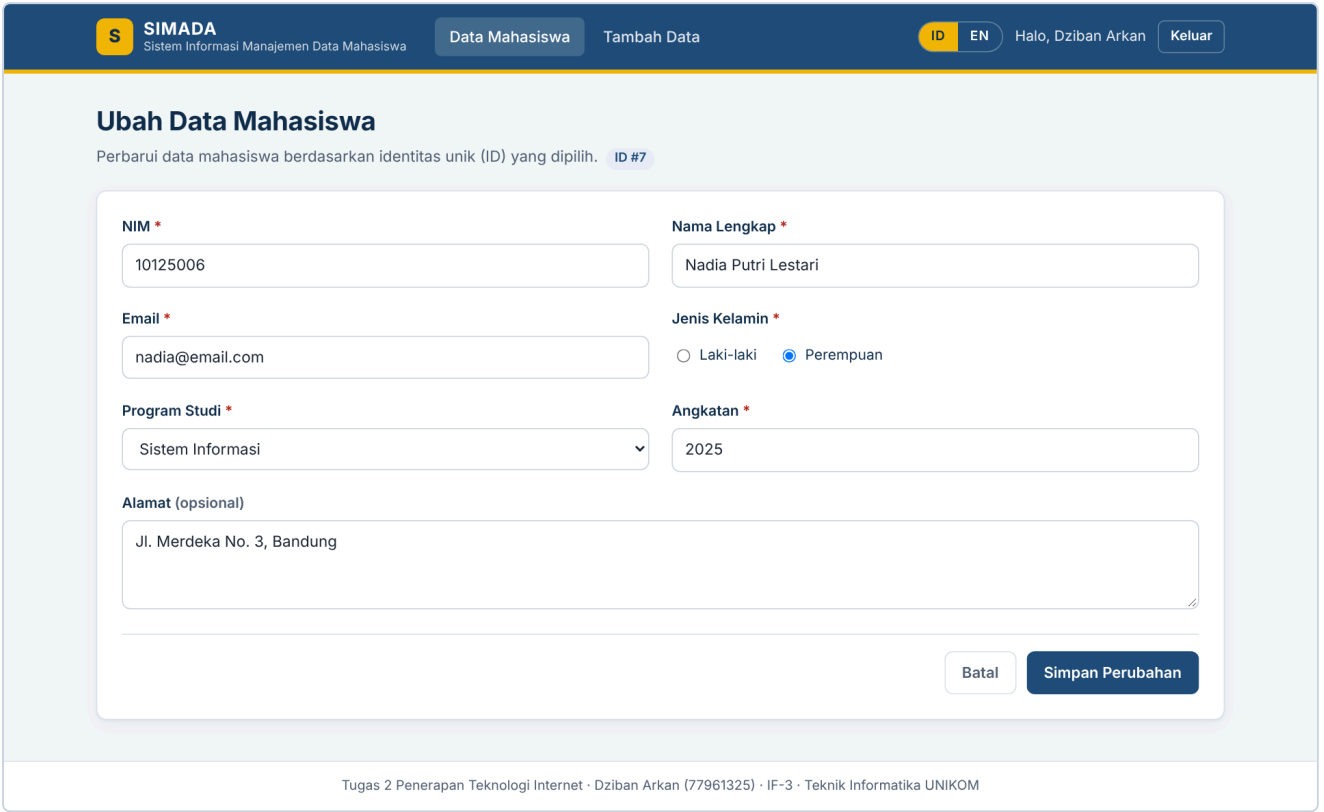


Gambar 4.7 Data berhasil disimpan ke database dan muncul di tabel.

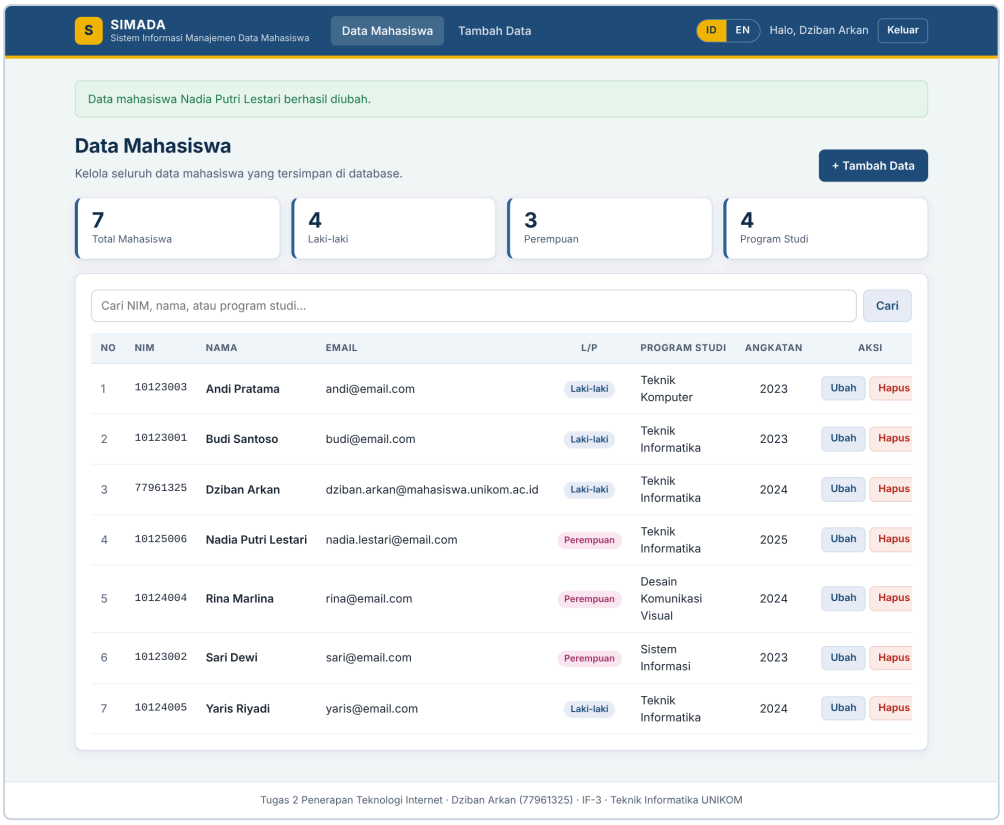


Gambar 4.8 Validasi NIM duplikat: NIM yang sudah terdaftar ditolak.

4.4 Update — Ubah Data

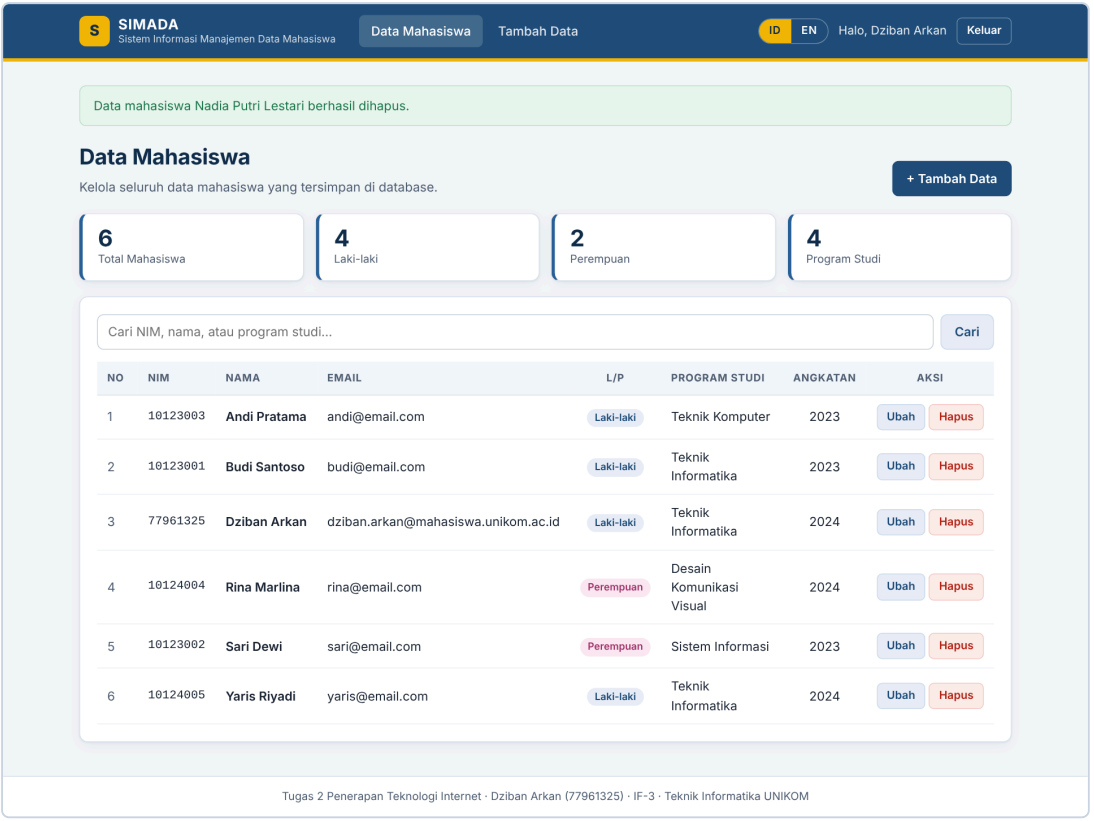


Gambar 4.9 Form ubah data terisi otomatis berdasarkan identitas unik (ID #7).



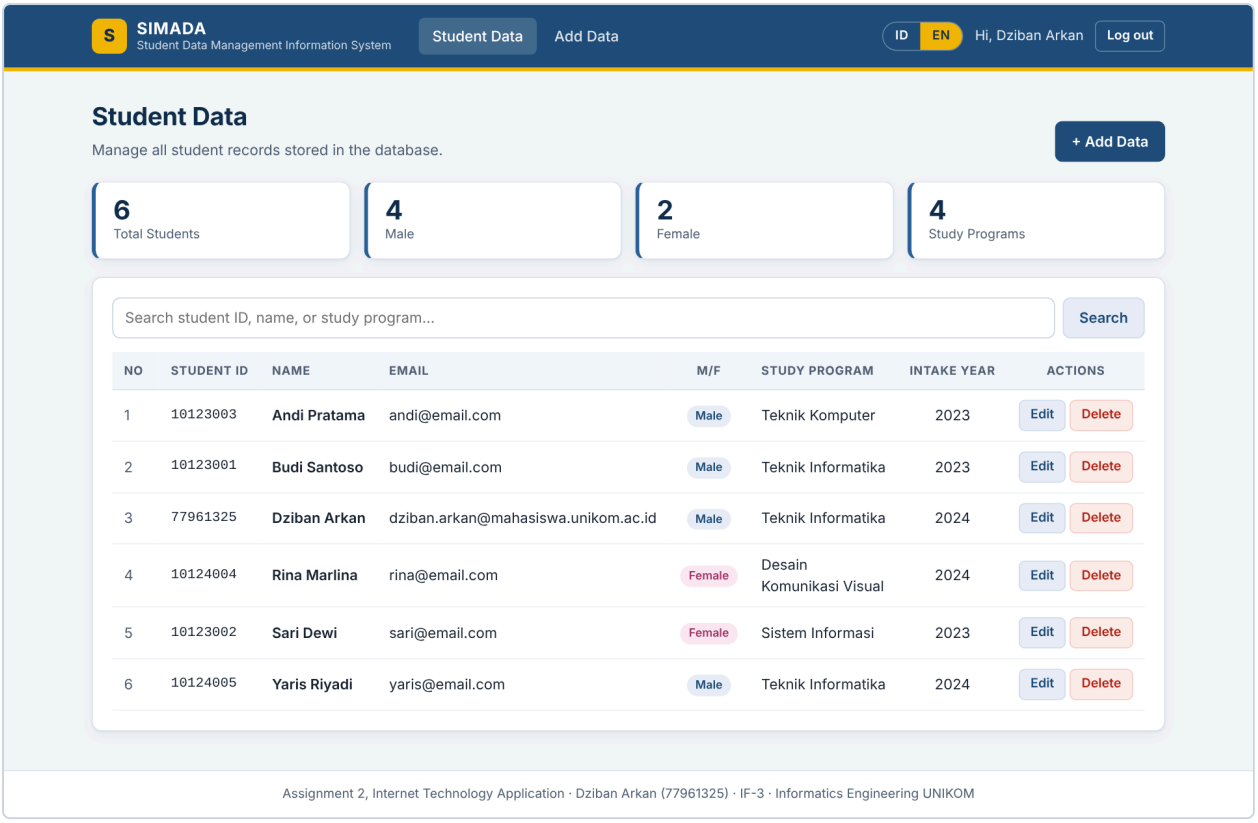
Gambar 4.10 Data berhasil diubah (email dan program studi diperbarui).

4.5 Delete — Hapus Data

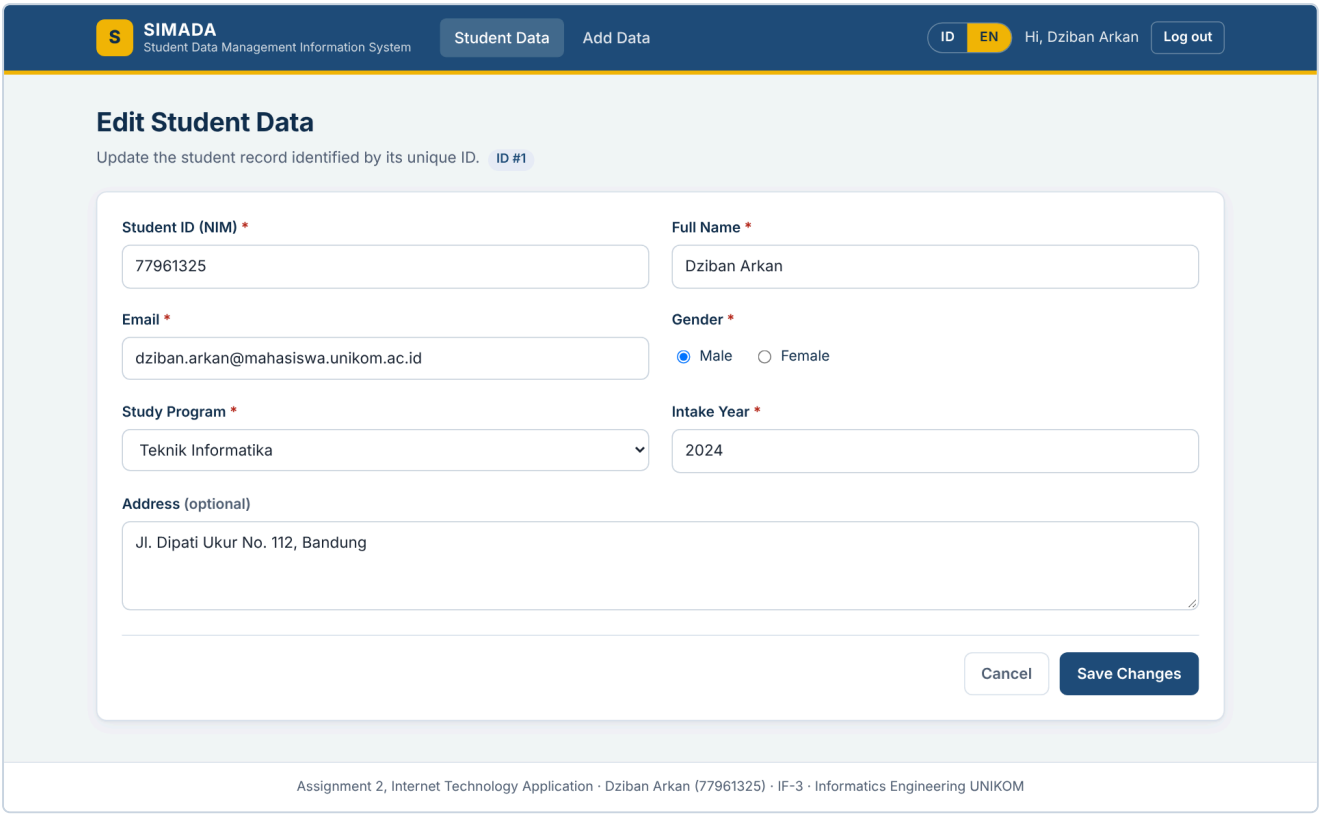


Gambar 4.11 Setelah konfirmasi, data berhasil dihapus dari database.

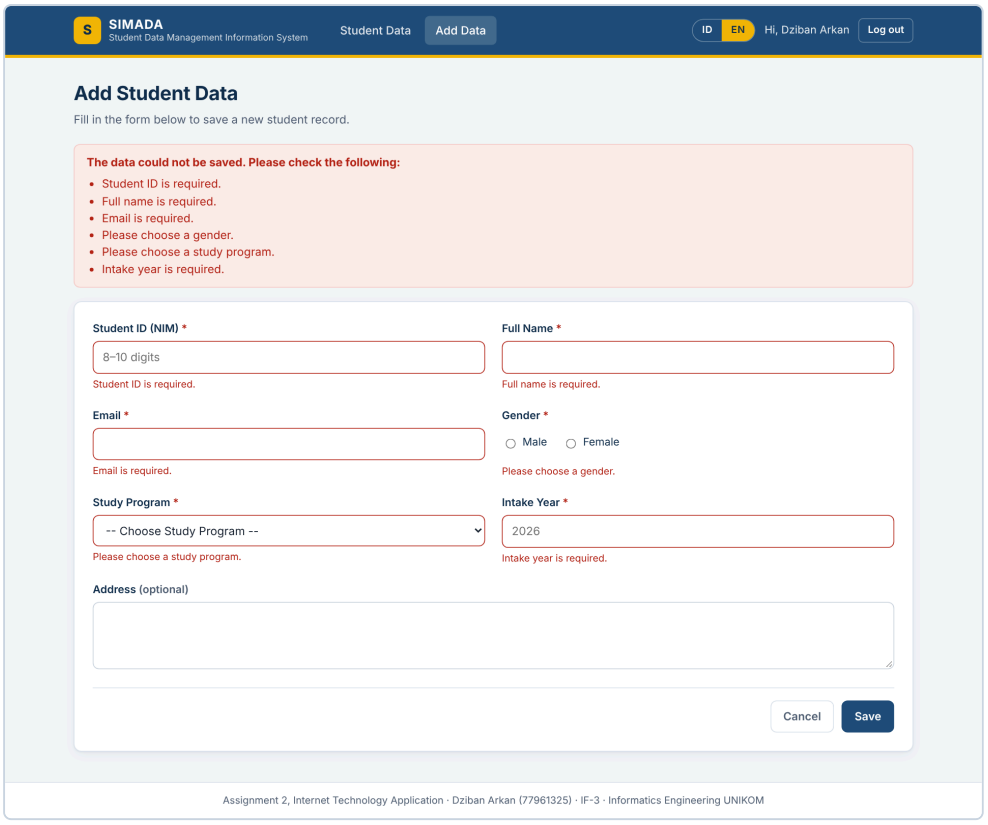
4.6 Fitur Dual Bahasa



Gambar 4.12 Setelah memilih **EN**, seluruh antarmuka berubah ke Bahasa Inggris.

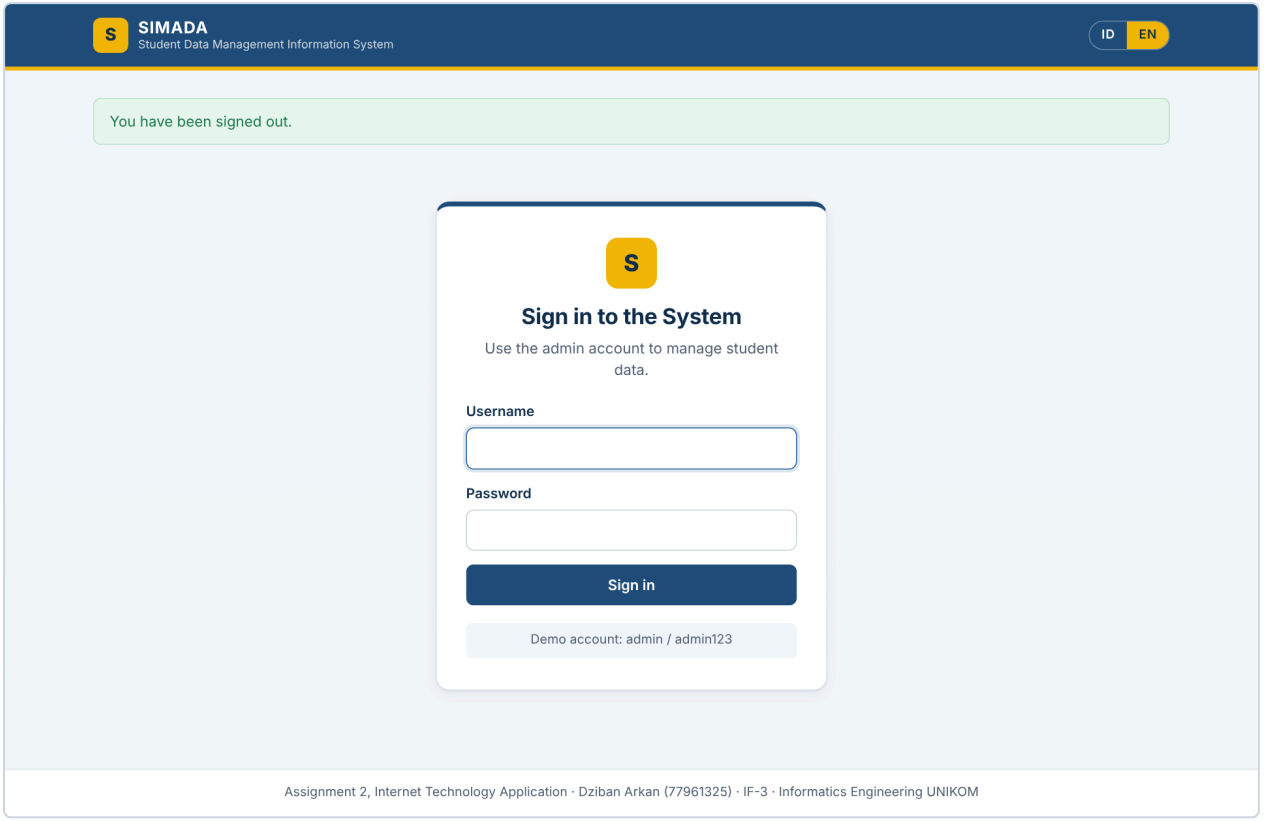


Gambar 4.13 Bahasa tetap Inggris saat berpindah ke halaman ubah data (tersimpan di `$_SESSION["lang"]`).

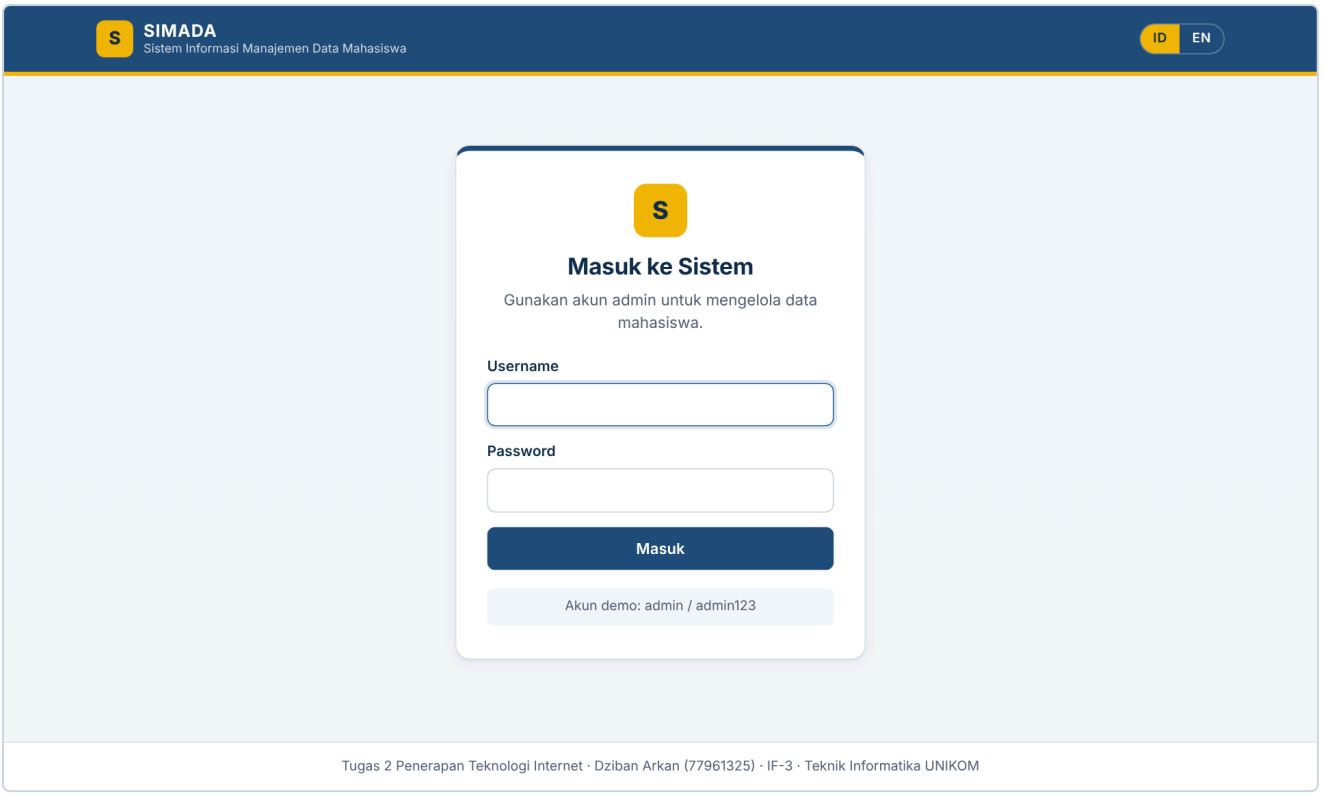


Gambar 4.14 Pesan validasi juga tampil dalam Bahasa Inggris.

4.7 Logout



Gambar 4.15 Logout: session login dihapus, pilihan bahasa tetap dipertahankan.



Gambar 4.16 Setelah logout, halaman CRUD kembali terkunci; bahasa dikembalikan ke Indonesia dari halaman login.

5. Kesimpulan

Aplikasi SIMADA telah memenuhi seluruh spesifikasi Tugas 2: form input dengan metode POST yang divalidasi menggunakan `switch` dan `if-else` serta disanitasi dengan `htmlspecialchars()` dan `filter_var()`; koneksi MySQL menggunakan PDO dengan operasi CRUD lengkap berbasis *prepared statement*; halaman CRUD yang dilindungi sistem login berbasis `$_SESSION`; serta fitur dual bahasa (Indonesia–Inggris) yang tersimpan di session sehingga konsisten di semua halaman.

Seluruh alur telah diuji secara langsung di browser: akses tanpa login, login gagal dan berhasil, tambah data valid dan tidak valid, NIM duplikat, pencarian, ubah data, hapus data, pergantian bahasa di beberapa halaman, dan logout. Semua pengujian berjalan sesuai harapan, seperti terlihat pada tangkapan layar di Bab 4.